



Tecnicatura universitaria en Programación

Trabajo Práctico Integrador (TPI)

Leandro Traficante - Comisión 01

Nadia Carolina Schnaible - Comisión 15

Fecha de entrega: 01.06.2026

Índice

Índice.....	2
Link GitHub - YouTube.....	4
Marco Teórico.....	4
Listas.....	4
Creación de listas en Python.....	4
Acceso y recorrido de listas.....	4
Métodos principales de las listas.....	5
Comparación y ordenamiento de listas.....	5
Diccionarios.....	5
Creación de diccionarios.....	6
Acceso a los elementos.....	6
Recorrer un diccionario.....	6
Agregar y modificar elementos.....	6
Eliminación de elementos.....	7
Cantidad de elementos.....	7
Verificación de claves.....	7
Comparación entre diccionarios.....	7
Diccionarios anidados.....	7
Objetos vista.....	8
Métodos principales de la clase dict.....	8
Funciones en Python.....	8
Sintaxis de una función.....	8
Uso y ventajas de las funciones.....	9
Parámetros y valores predeterminados.....	9
Funciones como objetos.....	9
Variables locales y globales.....	9
Funciones incorporadas en Python.....	10
Llamada de funciones.....	11
Modificación de objetos mutables.....	11
Condicionales en Python.....	12
Importancia de las condicionales.....	12
Declaración if.....	12
Valores considerados falsos en Python.....	12
Declaración else.....	13
Declaración elif.....	13
Declaración match.....	13
Características generales de las condicionales.....	14
Ordenamientos en Python.....	14
Orden ascendente y descendente.....	15
Ordenamiento personalizado con key.....	16
Ventajas de los ordenamientos en Python.....	16
Estadísticas básicas en Python.....	16

Media aritmética.....	16
Mediana.....	17
Moda.....	17
Varianza.....	17
Desviación estándar.....	17
Rango.....	18
Biblioteca statistics.....	18
Uso de estadísticas en programación.....	18
Importancia de la estadística en Python.....	19
Archivos CSV en Python.....	19
Características de los archivos CSV.....	19
Módulo csv en Python.....	19
Lectura de archivos CSV.....	20
Escritura de archivos CSV.....	20
Uso de DictReader y DictWriter.....	20
Lectura con DictReader.....	20
Escritura con DictWriter.....	21
Delimitadores en CSV.....	21
Ventajas del formato CSV.....	21
Limitaciones de los archivos CSV.....	21
Aplicaciones de los archivos CSV.....	21
Biblioteca Pandas y CSV.....	22
Decisiones técnicas y arquitectura.....	22
Arquitectura y Desarrollo del Programa.....	22
Imágenes de ejecución de Ejemplos.....	23
Diagrama de flujo de operaciones principales.....	25
Dificultades y Conclusiones.....	26
Dificultades.....	26
Conclusiones.....	27
Bibliografía.....	28

Link GitHub - YouTube

<https://github.com/nadiaschnaible/UTN-TUPaD-P1-TPI-Schnaible-Traficante/tree/main>
<https://youtu.be/8om5lKeMwEs>

Marco Teórico

Listas

Las listas son una de las estructuras de datos fundamentales en Python y se destacan por su flexibilidad y utilidad. Se trata de objetos mutables que permiten almacenar distintos tipos de datos dentro de una misma colección. En otros lenguajes de programación suelen conocerse como arrays, aunque en Python poseen características más avanzadas, ya que permiten modificar sus elementos y acceder a ellos mediante índices (El Pythonista, s.f.). En Python, las listas funcionan como estructuras doblemente enlazadas. Esto significa que cada elemento mantiene referencia tanto al elemento siguiente como al anterior, facilitando el recorrido y la manipulación de los datos.

Creación de listas en Python

Existen diferentes maneras de crear listas en Python. La más común es utilizando corchetes [], aunque también puede emplearse el constructor list(). Cuando se utiliza list() sobre un objeto iterable, Python genera una nueva lista con cada uno de los elementos contenidos en dicho iterable.

Acceso y recorrido de listas

Los elementos de una lista se acceden mediante índices numéricos. El primer elemento ocupa la posición 0, mientras que el último puede obtenerse utilizando el índice -1. Python también permite recorrer todos los elementos de una lista mediante estructuras iterativas como for.

Slicing o sublistas

Python incorpora una técnica llamada *slicing*, que permite extraer partes específicas de una lista de manera simple y eficiente. La sintaxis básica es:

[inicio:fin:paso]

Gracias a esta herramienta es posible obtener sublistas indicando una posición inicial, una final y un paso opcional. Tanto los índices como los pasos pueden ser positivos o negativos. Además, el slicing no solo sirve para consultar datos, sino también para modificar fragmentos completos de una lista.

Métodos principales de las listas

Las listas incluyen numerosos métodos integrados que facilitan la manipulación de datos. Algunos de los más utilizados son:

Método	Descripción	Ejemplo
append	Añade un elemento al final de la lista	<code>list.append(5)</code>
clear	Elimina toda la información de la lista	<code>list.clear()</code>
copy	Devuelve una copia de la lista	<code>list.copy()</code>
count	Devuelve el número de ocurrencias de un elemento en la lista	<code>list.count('p')</code>
extend	Extiende una lista extendida con otro iterador	<code>list.extend(lst2)</code>
index	Devuelve el índice que ocupa un elemento	<code>list.index('e')</code>
insert	Inserta un elemento en la lista en una posición exacta	<code>list.insert(3, 'y')</code>
pop	Elimina un elemento y lo devuelve de la posición marcada o del final	<code>list.pop(4)</code>
remove	Elimina la primera ocurrencia de un elemento de la lista	<code>list.remove('p')</code>
reverse	Invierte el orden de los elementos de la lista	<code>list.reverse()</code>
sort	Permite ordenar la lista con una función como key	<code>list.sort(key=lambda x: str(x) > '0')</code>

Imagen 1 - *Listas Python*. (s.f.). El Pythonista. <https://elpythonista.com/listas-python>

Comparación y ordenamiento de listas

Las listas pueden compararse utilizando operadores como `==`, `!=`, `<` o `>`. Estas comparaciones se realizan elemento por elemento. Cuando los datos son de tipos incompatibles, es necesario implementar comparaciones manuales.

Para ordenar listas se utiliza generalmente el método `sort()`, el cual puede complementarse con funciones `lambda` para definir criterios personalizados de ordenamiento.

Diccionarios

Los diccionarios en Python corresponden al tipo de dato `dict`, una estructura que permite relacionar claves con valores. A diferencia de otros tipos de datos secuenciales, como listas o tuplas, los diccionarios no utilizan índices numéricos para acceder a la información, sino claves únicas. Estas claves deben pertenecer a tipos de datos inmutables y hashables, como cadenas de texto, números enteros o tuplas (J2Logo, s.f.).

Un diccionario puede entenderse como una colección de pares “clave: valor”, donde cada clave identifica de forma única a un dato asociado. Gracias a esta característica, los diccionarios ofrecen un acceso rápido y eficiente a la información, incluso cuando contienen una gran cantidad de elementos.

Entre las principales características de los diccionarios se destacan las siguientes:

- Son mutables, por lo que sus elementos pueden modificarse luego de haber sido creados.
- Mantienen el orden de inserción de los elementos.
- Admiten distintos tipos de datos tanto en las claves como en los valores.

Creación de diccionarios

Existen diferentes formas de crear diccionarios en Python. La más utilizada consiste en escribir pares clave-valor entre llaves {} separados por comas.

```
d = {'uno': 1, 'dos': 2, 'tres': 3}
```

Acceso a los elementos

Para obtener el valor asociado a una clave, se utilizan corchetes junto con el nombre de la clave.

```
d['dos']
```

Si la clave no existe, Python genera una excepción llamada `KeyError`. Como alternativa más segura, puede utilizarse el método `get()`, que devuelve un valor predeterminado en caso de que la clave no esté presente.

```
d.get('cuatro', 4)
```

Recorrer un diccionario

Los diccionarios pueden recorrerse mediante bucles `for`. Es posible iterar únicamente sobre las claves, sobre los valores o sobre ambos elementos al mismo tiempo utilizando los métodos `keys()`, `values()` e `items()`.

```
for clave, valor in d.items():  
    print(clave, valor)
```

Agregar y modificar elementos

Debido a que los diccionarios son mutables, se pueden añadir nuevos elementos o modificar valores existentes mediante el operador de asignación.

```
d['cuatro'] = 4
```

Si la clave ya existe, el valor anterior será reemplazado por el nuevo.

Eliminación de elementos

Python ofrece distintos métodos para eliminar elementos de un diccionario:

- `pop()` elimina un elemento utilizando su clave.
- `popitem()` elimina el último elemento agregado.
- `del` elimina una clave específica.
- `clear()` elimina todos los elementos del diccionario.

```
d.pop('uno')
```

Cantidad de elementos

La función `len()` permite conocer cuántos pares clave-valor contiene un diccionario.

```
len(d)
```

Verificación de claves

El operador `in` se utiliza para comprobar si una clave existe dentro de un diccionario.

```
'uno' in d
```

Esto resulta útil para evitar errores antes de acceder o eliminar elementos.

Comparación entre diccionarios

Dos diccionarios son considerados iguales cuando contienen los mismos pares clave-valor, sin importar el orden en que se encuentren almacenados.

```
d1 == d2
```

Sin embargo, comparaciones como mayor o menor (`>` o `<`) no están permitidas entre diccionarios y generan un error de tipo `TypeError`.

Diccionarios anidados

Un diccionario también puede contener otros diccionarios como valores, formando estructuras conocidas como diccionarios anidados.

```
d = {'d1': {'k1': 1, 'k2': 2}}
```

Para acceder a un valor interno se utilizan índices encadenados.

```
d['d1']['k1']
```

Objetos vista

Los métodos `keys()`, `values()` e `items()` generan objetos vista dinámicos, los cuales reflejan automáticamente cualquier modificación realizada sobre el diccionario original.

Métodos principales de la clase dict

Método	Descripción
<code>clear()</code>	Elimina todos los elementos del diccionario.
<code>copy()</code>	Devuelve una copia poco profunda del diccionario.
<code>get(clave[, valor])</code>	Devuelve el valor de la <code>clave</code> . Si no existe, devuelve el valor <code>valor</code> si se indica y si no, <code>None</code> .
<code>items()</code>	Devuelve una vista de los pares <code>clave: valor</code> del diccionario.
<code>keys()</code>	Devuelve una vista de las claves del diccionario.
<code>pop(clave[, valor])</code>	Devuelve el valor del elemento cuya clave es <code>clave</code> y elimina el elemento del diccionario. Si la clave no se encuentra, devuelve <code>valor</code> si se proporciona. Si la clave no se encuentra y no se indica <code>valor</code> , lanza la excepción <code>KeyError</code> .
<code>popitem()</code>	Devuelve un par (<code>clave, valor</code>) aleatorio del diccionario. Si el diccionario está vacío, lanza la excepción <code>KeyError</code> .
<code>setdefault(clave[, valor])</code>	Si la <code>clave</code> está en el diccionario, devuelve su valor. Si no lo está, inserta la <code>clave</code> con el valor <code>valor</code> y lo devuelve (si no se especifica <code>valor</code> , por defecto es <code>None</code>).
<code>update(iterable)</code>	Actualiza el diccionario con los pares <code>clave: valor</code> del <code>iterable</code> .
<code>values()</code>	Devuelve una vista de los valores del diccionario.

Imagen 2 - *Tipo dict en Python* [Imagen]. (s.f.). J2Logo. <https://j2logo.com/python/tutorial/tipo-dict-python/>

Funciones en Python

Las funciones en Python son bloques de código reutilizables que permiten organizar y simplificar programas. Su principal ventaja es que facilitan la reutilización de instrucciones sin necesidad de escribir el mismo código varias veces. Gracias a esto, los programas resultan más ordenados, fáciles de mantener y menos propensos a errores.

Python incluye numerosas funciones integradas, como `print()` o `len()`, aunque también permite que los programadores creen sus propias funciones según las necesidades de cada proyecto (freeCodeCamp, s.f.).

Sintaxis de una función

Para definir una función en Python se utiliza la palabra clave `def`, seguida del nombre de la función y paréntesis que pueden incluir parámetros. Luego se colocan dos puntos `:` y un

bloque de código indentado.

```
def diHola():  
    print("Hello!")
```

Las funciones pueden recibir parámetros y también devolver resultados mediante la palabra clave return.

```
def multiplica(val1, val2):  
    return val1 * val2
```

Uso y ventajas de las funciones

El uso de funciones favorece la modularidad del código y permite desarrollar soluciones más claras y eficientes. Además, ayudan a reducir la cantidad de líneas de programación y facilitan la depuración de errores.

Cada vez que una función es llamada, Python ejecuta el bloque de instrucciones asociado a ella.

```
resultado = multiplica(3, 5)
```

Parámetros y valores predeterminados

Las funciones pueden incluir parámetros con valores predeterminados. De esta manera, si no se proporciona un argumento, Python utilizará automáticamente el valor definido.

```
def suma(a, b=3):  
    return a + b
```

También es posible enviar argumentos utilizando el nombre del parámetro.

```
suma(b=2, a=2)
```

Funciones como objetos

En Python, las funciones son tratadas como objetos. Esto significa que pueden asignarse a variables y utilizarse posteriormente como si fueran la función original.

```
s = suma  
resultado = s(1, 2)
```

Variables locales y globales

Las variables creadas dentro de una función solo existen en el ámbito de dicha función. Estas se conocen como variables locales.

```
def duplica(num):
```

```
    x = num * 2
```

```
    return x
```

Por otro lado, las variables definidas fuera de una función se consideran globales y pueden ser utilizadas dentro de ella, aunque no modificadas directamente sin usar la palabra clave `global`.

Funciones incorporadas en Python

Python incorpora numerosas funciones predefinidas que facilitan distintas tareas de programación.

Función `max()`

La función `max()` devuelve el valor más grande entre varios elementos o dentro de un iterable.

```
max(2, 3, 23)
```

Función `min()`

`min()` realiza la operación opuesta, devolviendo el valor más pequeño.

```
min([1, 2, 4, -54])
```

Función `divmod()`

La función `divmod()` devuelve el cociente y el resto de una división.

```
divmod(5, 2)
```

Función `hex()`

`hex()` convierte un número entero en una representación hexadecimal.

```
hex(16)
```

Función `len()`

La función `len()` permite conocer la cantidad de elementos de un objeto, como listas, cadenas o diccionarios.

```
len("basketball")
```

Funciones `ord()` y `chr()`

- `ord()` transforma un carácter en su código Unicode.
- `chr()` realiza la operación inversa, convirtiendo un código Unicode en un carácter.

```
ord('d')
```

```
chr(49)
```

Función `input()`

La función `input()` se utiliza para recibir datos ingresados por el usuario mediante teclado. Cuando esta función es llamada, la ejecución del programa se detiene hasta que el usuario ingresa información.

```
nombre = input("Ingrese su nombre: ")
```

Generalmente, los datos recibidos mediante `input()` son almacenados como cadenas de texto.

Llamada de funciones

Definir una función no implica ejecutarla automáticamente. Para que el bloque de código se ejecute, es necesario llamar a la función utilizando su nombre y los argumentos correspondientes.

```
def di_hola():  
    print("Hello")
```

```
di_hola()
```

Modificación de objetos mutables

Aunque los argumentos enviados a una función no pueden reasignarse fuera de ella, sí es posible modificar objetos mutables, como las listas.

```
def fn(arg):  
    arg.append(1)
```

Esto significa que los cambios realizados sobre listas o diccionarios dentro de una función permanecerán luego de finalizar la ejecución.

Condicionales en Python

Las estructuras condicionales en Python permiten que un programa tome decisiones durante su ejecución. Gracias a ellas, el flujo del código puede seguir diferentes caminos dependiendo del resultado de determinadas condiciones o expresiones lógicas. Este mecanismo resulta fundamental para desarrollar programas dinámicos y adaptables a distintas situaciones (Coder, s.f.).

Las condicionales funcionan evaluando expresiones booleanas. Según si el resultado es verdadero (True) o falso (False), Python ejecutará un bloque específico de instrucciones.

Importancia de las condicionales

En programación, las condicionales brindan control sobre el comportamiento del programa. Esto permite que ciertas acciones solo se ejecuten cuando se cumplen determinadas condiciones. De esta manera, los programas pueden responder de forma distinta según los datos ingresados o el contexto de ejecución.

Python dispone de varias estructuras condicionales principales:

- if
- else
- elif
- match

Cada una cumple una función específica dentro de la lógica de decisión.

Declaración if

La estructura `if` se utiliza para evaluar una condición. Si la expresión evaluada resulta verdadera, el bloque de código indentado se ejecutará.

```
n = int(input("Choose number: "))
```

```
if n % 2 == 0:  
    print("The number is even")
```

En este ejemplo, el programa verifica si el número ingresado es divisible por 2. Si la condición se cumple, se imprime un mensaje indicando que el número es par.

Valores considerados falsos en Python

Python convierte automáticamente ciertos valores a booleanos. Algunos valores son interpretados como falsos por defecto:

- False
- None
- 0
- 0.0
- "" (cadena vacía)
- [] (lista vacía)
- () (tupla vacía)

- {} (diccionario vacío)
- set() (conjunto vacío)

Cualquier otro valor generalmente será interpretado como verdadero.

Declaración else

La declaración else complementa a if y se ejecuta cuando la condición principal resulta falsa.

```
n = int(input("Choose number: "))
```

```
if n % 2 == 0:
    print("The number is even")
else:
    print("The number is odd")
```

En este caso, si el número no es par, el programa mostrará que es impar.

Declaración elif

La estructura elif permite agregar condiciones adicionales dentro de una misma evaluación lógica. Si la condición del if inicial no se cumple, Python verificará las condiciones definidas en los bloques elif.

```
alice_age = int(input("Choose number: "))

if alice_age < 18:
    print("Alice is younger than 18 years.")
elif 18 <= alice_age <= 21:
    print("Alice has between 18 and 21 years.")
else:
    print("Alice has more than 21 years.")
```

El uso de elif facilita la organización de múltiples escenarios sin necesidad de crear varios bloques if independientes.

Declaración match

A partir de Python 3.10 se incorporó la declaración match, utilizada para realizar comparaciones entre un valor y distintos casos posibles.

A diferencia de if y elif, que trabajan mediante expresiones booleanas, match compara directamente un valor con diferentes patrones definidos.

```
from datetime import datetime
```

```
currentDay = datetime.now().weekday()
```

```
match currentDay:
    case 0:
        day = "Sunday"
    case 1:
        day = "Monday"
    case 2:
        day = "Tuesday"
    case 3:
        day = "Wednesday"
    case 4:
        day = "Thursday"
    case 5:
        day = "Friday"
    case 6:
        day = "Saturday"
    case _:
        day = "Invalid option"

print(day)
```

Esta estructura resulta especialmente útil cuando se necesita comparar una variable contra múltiples valores específicos, mejorando la legibilidad del código.

Características generales de las condicionales

Las estructuras condicionales en Python presentan varias ventajas:

- Permiten crear programas más dinámicos.
- Facilitan la toma de decisiones automáticas.
- Mejoran la organización y claridad del código.
- Ayudan a controlar distintos escenarios posibles dentro de un programa.

Además, la indentación en Python cumple un rol fundamental, ya que determina qué instrucciones pertenecen a cada bloque condicional.

Ordenamientos en Python

El ordenamiento de datos es una de las operaciones más importantes dentro de la programación, ya que permite organizar elementos siguiendo un criterio específico, como orden ascendente o descendente. En Python, el ordenamiento puede realizarse de manera sencilla mediante funciones y métodos integrados, especialmente `sorted()` y `sort()` (Python Documentation, 2026).

Los algoritmos de ordenamiento son fundamentales para mejorar la búsqueda, el análisis y la manipulación de datos. Python implementa internamente algoritmos optimizados capaces de ordenar listas y otros objetos iterables de forma eficiente.

Función `sorted()`

La función `sorted()` devuelve una nueva lista ordenada a partir de cualquier objeto iterable, sin modificar el original.

```
numeros = [5, 2, 3, 1, 4]
```

```
resultado = sorted(numeros)
```

```
print(resultado)
```

Resultado: [1, 2, 3, 4, 5]

Una de las principales ventajas de `sorted()` es que puede utilizarse con diferentes tipos de iterables, como listas, tuplas, cadenas o diccionarios.

Método `sort()`

El método `sort()` pertenece exclusivamente a las listas y modifica directamente la lista original.

```
numeros = [5, 2, 3, 1, 4]
```

```
numeros.sort()
```

```
print(numeros)
```

A diferencia de `sorted()`, este método no devuelve una nueva lista, sino que reorganiza los elementos existentes.

Orden ascendente y descendente

Por defecto, Python ordena los elementos de menor a mayor. Sin embargo, utilizando el parámetro `reverse=True`, es posible invertir el orden.

```
numeros.sort(reverse=True)
```

Esto produce un orden descendente.

Ordenamiento personalizado con key

Python permite personalizar el criterio de ordenamiento mediante el parámetro `key`. Este parámetro recibe una función que indica cómo deben compararse los elementos.

```
palabras = ["perro", "gato", "elefante"]
```

```
resultado = sorted(palabras, key=len)
```

```
print(resultado)
```

Ventajas de los ordenamientos en Python

Entre las principales ventajas de las herramientas de ordenamiento de Python se encuentran:

- Alta eficiencia.
- Simplicidad de uso.
- Estabilidad del algoritmo.
- Posibilidad de personalizar criterios de ordenamiento.
- Compatibilidad con múltiples estructuras de datos.

Gracias a estas características, Python es ampliamente utilizado para procesamiento de datos, análisis estadístico y desarrollo de aplicaciones.

Estadísticas básicas en Python

La estadística básica es una rama de las matemáticas que permite recolectar, organizar, analizar e interpretar datos. En programación, especialmente en Python, las herramientas estadísticas son ampliamente utilizadas para el análisis de información, ciencia de datos, inteligencia artificial y visualización de resultados.

Python incluye bibliotecas y funciones integradas que facilitan la realización de cálculos estadísticos de forma rápida y eficiente. Entre las operaciones más comunes se encuentran el cálculo de media, mediana, moda, desviación estándar y varianza (Python Documentation, 2026).

Media aritmética

La media aritmética, también conocida como promedio, se obtiene sumando todos los valores y dividiendo el resultado entre la cantidad total de elementos.

En Python, puede calcularse mediante el módulo `statistics`.

```
import statistics
```

```
datos = [10, 20, 30, 40]
```

```
resultado = statistics.mean(datos)
```

```
print(resultado)
```

Resultado: 25

Mediana

La mediana corresponde al valor central de un conjunto de datos ordenados. Si la cantidad de elementos es par, se calcula el promedio de los dos valores centrales.

```
import statistics
```

```
datos = [10, 20, 30, 40, 50]  
resultado = statistics.median(datos)
```

```
print(resultado)
```

La mediana es especialmente útil cuando existen valores extremos que podrían alterar significativamente la media.

Moda

La moda es el valor que más veces aparece dentro de un conjunto de datos.

```
import statistics
```

```
datos = [1, 2, 2, 3, 4]  
resultado = statistics.mode(datos)
```

```
print(resultado)
```

En este ejemplo, el valor 2 corresponde a la moda porque aparece con mayor frecuencia.

Varianza

La varianza mide qué tan dispersos están los datos respecto a la media. Cuanto mayor sea la varianza, mayor será la dispersión de los valores.

```
import statistics
```

```
datos = [10, 20, 30, 40]  
resultado = statistics.variance(datos)
```

```
print(resultado)
```

La varianza resulta útil para comprender la distribución de los datos.

Desviación estándar

La desviación estándar es una medida estadística relacionada con la varianza. Indica cuánto se alejan los datos del promedio.

```
import statistics
```

```
datos = [10, 20, 30, 40]  
resultado = statistics.stdev(datos)
```

```
print(resultado)
```

Una desviación estándar baja indica que los valores están cercanos a la media, mientras que una desviación alta refleja mayor dispersión.

Rango

El rango representa la diferencia entre el valor máximo y el valor mínimo de un conjunto de datos.

```
datos = [10, 20, 30, 40]
rango = max(datos) - min(datos)

print(rango)
```

El rango proporciona una medida simple de dispersión.

Biblioteca statistics

Python incluye la biblioteca estándar statistics, diseñada para realizar cálculos estadísticos básicos sin necesidad de instalar paquetes externos.

Entre sus funciones más importantes se encuentran:

Función	Descripción
mean()	Calcula la media
median()	Calcula la mediana
mode()	Calcula la moda
variance()	Calcula la varianza
stdev()	Calcula la desviación estándar

Uso de estadísticas en programación

Las estadísticas básicas tienen múltiples aplicaciones en programación y análisis de datos:

- Interpretación de grandes volúmenes de información.
- Detección de patrones y tendencias.
- Desarrollo de modelos predictivos.
- Análisis financiero y científico.
- Machine Learning e inteligencia artificial.

Python se ha convertido en uno de los lenguajes más utilizados en análisis estadístico debido a su simplicidad y la gran cantidad de bibliotecas especializadas disponibles.

Importancia de la estadística en Python

El uso de herramientas estadísticas permite transformar datos en información útil para la toma de decisiones. Gracias a bibliotecas como statistics, NumPy y Pandas, Python facilita enormemente el procesamiento y análisis de datos.

Además, la combinación entre programación y estadística resulta fundamental en áreas como:

- Ciencia de datos.
- Investigación científica.
- Economía.
- Medicina.
- Ingeniería.
- Marketing digital.

Archivos CSV en Python

Los archivos CSV (Comma-Separated Values) son uno de los formatos más utilizados para almacenar e intercambiar datos estructurados. Este tipo de archivo organiza la información en filas y columnas, donde cada valor se encuentra separado por comas u otros delimitadores. Los archivos CSV son ampliamente empleados en bases de datos, hojas de cálculo y análisis de datos debido a su simplicidad y compatibilidad con numerosos programas (Python Documentation, 2026).

En Python, el manejo de archivos CSV puede realizarse mediante el módulo incorporado csv, el cual facilita la lectura, escritura y manipulación de este tipo de archivos.

Características de los archivos CSV

Los archivos CSV presentan varias características importantes:

- Son archivos de texto plano.
- Organizan los datos en forma tabular.
- Cada línea representa una fila de datos.
- Los valores suelen separarse mediante comas.
- Son compatibles con programas como Excel y Google Sheets.

Ejemplo de un archivo CSV:

```
nombre,edad,ciudad  
Ana,25,Madrid  
Luis,30,Berlín  
Sofía,28,Buenos Aires
```

Módulo csv en Python

Python incluye el módulo estándar csv, diseñado específicamente para trabajar con archivos CSV.

Para utilizarlo, primero debe importarse: `import csv`

Este módulo proporciona herramientas para leer y escribir datos de manera sencilla.

Lectura de archivos CSV

Para leer un archivo CSV se utiliza generalmente la función `csv.reader()`.

```
import csv
with open('datos.csv', newline='') as archivo:
    lector = csv.reader(archivo)

    for fila in lector:
        print(fila)
```

En este ejemplo, cada fila del archivo se almacena como una lista.

Escritura de archivos CSV

El módulo `csv` también permite crear y escribir archivos CSV mediante `csv.writer()`.

```
import csv

with open('datos.csv', 'w', newline='') as archivo:
    escritor = csv.writer(archivo)

    escritor.writerow(['Nombre', 'Edad'])
    escritor.writerow(['Ana', 25])
```

Cada llamada a `writerow()` agrega una nueva fila al archivo.

Uso de DictReader y DictWriter

Python ofrece herramientas más avanzadas llamadas `DictReader` y `DictWriter`, las cuales permiten trabajar con los datos utilizando diccionarios.

Lectura con DictReader

```
import csv

with open('datos.csv') as archivo:
    lector = csv.DictReader(archivo)

    for fila in lector:
        print(fila['Nombre'])
```

Cada fila se interpreta como un diccionario donde las claves corresponden a los nombres de las columnas.

Escritura con DictWriter

```
import csv
```

```
with open('datos.csv', 'w', newline='') as archivo:
```

```
    campos = ['Nombre', 'Edad']
```

```
    escritor = csv.DictWriter(archivo, fieldnames=campos)
```

```
    escritor.writeheader()
```

```
    escritor.writerow({'Nombre': 'Luis', 'Edad': 30})
```

Delimitadores en CSV

Aunque el delimitador más común es la coma, algunos archivos CSV utilizan otros caracteres como punto y coma (;) o tabulaciones.

Python permite especificar el delimitador manualmente:

```
csv.reader(archivo, delimiter=';')
```

Esto resulta útil para adaptarse a distintos formatos de datos.

Ventajas del formato CSV

Entre las principales ventajas de los archivos CSV se destacan:

- Facilidad de lectura y escritura.
- Compatibilidad con múltiples plataformas.
- Tamaño reducido de almacenamiento.
- Simplicidad en el intercambio de datos.
- Integración con herramientas de análisis y bases de datos.

Limitaciones de los archivos CSV

A pesar de sus ventajas, los archivos CSV también presentan algunas limitaciones:

- No almacenan formatos ni estilos.
- No admiten relaciones complejas entre datos.
- No poseen mecanismos de seguridad integrados.
- Pueden presentar problemas cuando los datos contienen comas o caracteres especiales.

Por esta razón, en aplicaciones más complejas suelen utilizarse bases de datos o formatos más avanzados como JSON o XML.

Aplicaciones de los archivos CSV

Los archivos CSV son ampliamente utilizados en distintas áreas:

- Ciencia de datos.
- Análisis estadístico.
- Exportación de bases de datos.

- Sistemas administrativos.
- Machine Learning.
- Automatización de reportes.

Python facilita enormemente el procesamiento de este tipo de archivos gracias a sus bibliotecas integradas y externas.

Biblioteca Pandas y CSV

Además del módulo csv, una de las herramientas más utilizadas para trabajar con archivos CSV es la biblioteca Pandas.

Con Pandas, la lectura de archivos CSV se simplifica considerablemente:

```
import pandas as pd
df = pd.read_csv('datos.csv')
```

```
print(df)
```

Esta biblioteca permite realizar análisis avanzados y manipular grandes volúmenes de datos de manera eficiente.

Decisiones técnicas y arquitectura

Arquitectura y Desarrollo del Programa

El desarrollo del programa comenzó a partir del análisis de las consignas planteadas para el trabajo práctico. En primer lugar, se realizó una planificación general de las funcionalidades necesarias, definiendo qué operaciones debía realizar el sistema de gestión de países y cómo se organizarían dentro de un menú principal.

A partir de esta planificación se decidió estructurar el programa utilizando funciones y una lista de diccionarios para almacenar la información de cada país. Cada diccionario contiene los datos principales requeridos por la consigna: nombre, población, superficie y continente. Además, se implementó el uso de archivos CSV para permitir la persistencia de los datos, de manera que la información pudiera conservarse incluso después de cerrar el programa. La arquitectura general del código se organizó en diferentes etapas. Primero, se desarrolló una función encargada de cargar los datos desde el archivo países.csv utilizando la biblioteca csv. Luego, dentro de la función principal (main()), se implementó un menú interactivo con un bucle while, permitiendo que el usuario pudiera ejecutar múltiples operaciones sin reiniciar el programa.

Cada opción del menú fue desarrollada de manera independiente siguiendo una lógica modular. Entre las funcionalidades implementadas se encuentran:

- Agregar nuevos países.
- Actualizar población y superficie.
- Buscar países por nombre.
- Filtrar países según diferentes criterios.
- Ordenar registros.

- Mostrar estadísticas generales.

Durante la etapa de escritura del código se incorporaron validaciones para evitar errores de ingreso de datos, utilizando estructuras try-except y verificaciones de campos vacíos.

También se implementaron búsquedas parciales y normalización de texto para mejorar el funcionamiento del sistema.

Posteriormente se realizó una etapa de pruebas, en la cual se verificó el comportamiento de cada opción del menú utilizando distintos casos posibles, incluyendo datos incorrectos, nombres repetidos, búsquedas sin coincidencias y filtros con rangos inválidos. Estas pruebas permitieron detectar errores lógicos y mejorar la estabilidad general del programa.

Finalmente, se llegó a una propuesta funcional capaz de gestionar datos de países mediante operaciones de carga, búsqueda, modificación y filtrado, manteniendo una estructura organizada y comprensible. El desarrollo del trabajo permitió aplicar conceptos fundamentales de programación en Python, manejo de archivos CSV, estructuras de datos, validación de entradas y modularización del código.

Imágenes de ejecución de Ejemplos

```

--- Menú ---
1) Agregar país
2) Actualizar población y superficie de un país
3) Buscar país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar estadísticas
0) Salir
Opción: 3

--- Busque un país por su nombre ---
Ingrese el país que quiere buscar: arg

País encontrado:
Nombre: Argentina
Población: 45892285
Superficie: 2780000
Continente: América

--- Menú ---
1) Agregar país
2) Actualizar población y superficie de un país
3) Buscar país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar estadísticas
0) Salir
Opción: █

```

En esta imagen se puede ver como el programa funciona tanto con la búsqueda parcial o total, haciendo coincidir la búsqueda arg, con la palabra Argentina.

```

--- Menú ---
1) Agregar país
2) Actualizar población y superficie de un país
3) Buscar país por nombre
4) Filtrar países
5) Ordenar países
6) Mostrar estadísticas
0) Salir
Opción: 4

--- Filtrar países ---
1) Por Continente:
2) Por rango de población
3) Por rango de superficie
0) Para salir del menú
Opción: 1
Ingrese el continente: america
País: Argentina
País: Brasil
País: Estados Unidos
País: México

--- Filtrar países ---
1) Por Continente:
2) Por rango de población
3) Por rango de superficie
0) Para salir del menú
Opción: █

```

En esta imagen, se puede ver que el programa filtra la búsqueda por nombre, reconociendo la palabra america, que en el archivo csv esta escrita como corresponde, América.

```

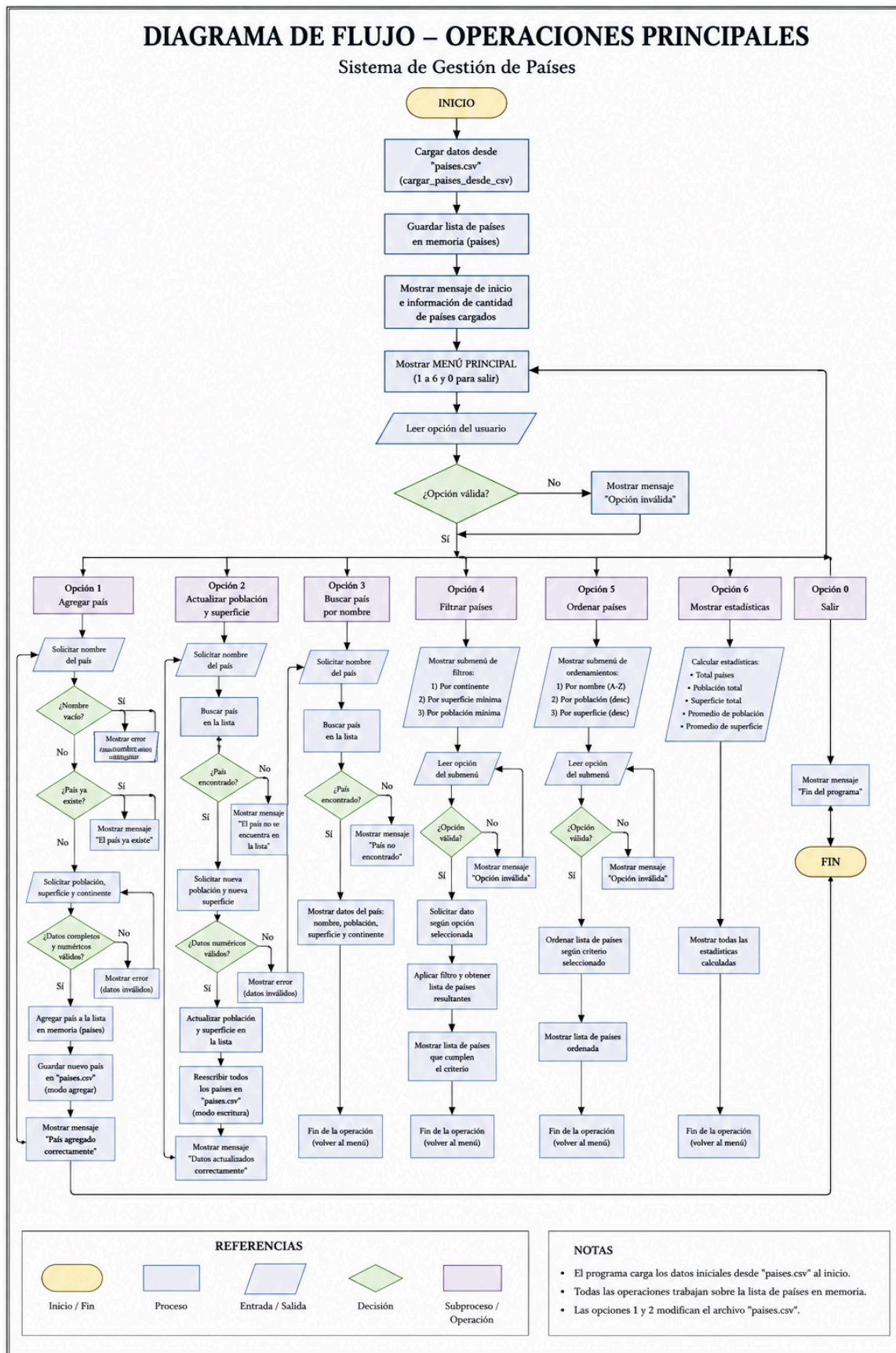
--- Filtrar países ---
1) Por Continente:
2) Por rango de población
3) Por rango de superficie
0) Para salir del menú
Opción: 2
Población mínima: 5000000
Población máxima: 80000000
Argentina - 45892285
España - 47415750
Francia - 64773000
Australia - 26100000

--- Filtrar países ---
1) Por Continente:
2) Por rango de población
3) Por rango de superficie
0) Para salir del menú
Opción: █

```

En esta imagen, se puede observar como el programa puede filtrar los países de acuerdo al rango de la población ingresada, devolviendo los países que se encuentran dentro de las cifras ingresadas.

Diagrama de flujo de operaciones principales



Dificultades y Conclusiones

Dificultades

Durante el desarrollo del trabajo práctico surgieron distintas dificultades relacionadas principalmente con la validación y búsqueda de datos dentro del programa.

Una de las principales complicaciones fue implementar correctamente las búsquedas parciales de países. En un primer momento, la búsqueda solo funcionaba cuando el nombre ingresado coincidía exactamente con el almacenado en la lista. Luego de varias pruebas y modificaciones en las condiciones del programa, se logró que el sistema pudiera encontrar coincidencias parciales, mejorando la funcionalidad y la experiencia del usuario.

Otra dificultad importante apareció en el punto 4 del menú, correspondiente al filtrado de países. El programa inicialmente no reconocía correctamente palabras escritas con acentos ni diferencias entre mayúsculas y minúsculas. Por ejemplo, si un continente estaba almacenado con tilde, la búsqueda podía fallar dependiendo de cómo el usuario escribiera el texto.

Para resolver este problema se investigaron distintas alternativas y finalmente se decidió utilizar la biblioteca de Python `unicodedata`, mediante `import unicodedata`, que permitió normalizar los textos y eliminar diferencias relacionadas con acentos y caracteres especiales. Esta solución requirió investigación adicional, ya que dicha biblioteca no había sido trabajada previamente en clase. Antes de llegar a esta implementación se probaron otros métodos que no resultaron completamente efectivos o generaban código más complejo.

Además, durante el desarrollo del programa se observó una dificultad personal relacionada con la estructura del código. En varias ocasiones se implementaron soluciones demasiado extensas o repetitivas y, posteriormente, se descubrieron formas más simples, breves y eficientes de resolver el mismo problema. Esto permitió comprender la importancia de la optimización y de planificar mejor la lógica antes de programar.

Conclusiones

En términos de cierre, el trabajo práctico permite consolidar de forma clara la integración entre conceptos teóricos de programación y su aplicación concreta en un sistema funcional desarrollado en Python.

A partir de las consignas iniciales, se llevó a cabo una planificación estructurada que definió la arquitectura general del programa, organizando la lógica en módulos y funciones con responsabilidades bien delimitadas. Esto facilitó tanto la lectura del código como su mantenimiento y escalabilidad, evitando una estructura monolítica difícil de gestionar.

El proceso de desarrollo incluyó la escritura progresiva del código, acompañado por pruebas iterativas que permitieron detectar y corregir errores en distintas etapas. En particular, el manejo de archivos CSV, las búsquedas parciales y los filtros por criterios múltiples representaron puntos críticos que requirieron ajustes y refinamiento lógico hasta alcanzar un funcionamiento estable.

El diagrama de flujo aportó una visión estratégica del comportamiento del sistema, funcionando como puente entre la idea abstracta y la implementación real.

En conjunto con los contenidos teóricos (condicionales, estructuras de control, ordenamiento y manejo de datos), permitió sostener decisiones de diseño más coherentes y justificadas.

Como resultado, se obtuvo una solución funcional, organizada y alineada con los requerimientos planteados.

El desarrollo evidenció que la programación no es solo escritura de código, sino también toma de decisiones, iteración constante y optimización progresiva.

Bibliografía

El Pythonista. (s.f.). *Listas Python*. El Pythonista. <https://elpythonista.com/listas-python>

J2Logo. (s.f.). *Tipo dict en Python*. J2Logo <https://j2logo.com/python/tutorial/tipo-dict-python/>

FreeCodeCamp. (s.f.). *Guía de funciones de Python con ejemplos*. freeCodeCamp. <https://www.freecodecamp.org/espanol/news/guia-de-funciones-de-python-con-ejemplos/>

Coder, D. (s.f.). *Condicionales en Python: if, else y match*. Medium. <https://medium.com/@diego.coder/condicionales-en-python-if-else-y-match-78490a8eb304>

Dalke, A., & Hettinger, R. (2026). *Sorting Techniques*. Python Documentation. <https://docs.python.org/3/howto/sorting.html>

Python Software Foundation. (2026). *statistics — Mathematical statistics functions*. Python Documentation. <https://docs.python.org/3/library/statistics.html>

GeeksforGeeks. (2026). *Python statistics module*. GeeksforGeeks. <https://www.geeksforgeeks.org/python-statistics-module/>

Python Software Foundation. (2026). *csv — CSV File Reading and Writing*. Python Documentation. <https://docs.python.org/3/library/csv.html>

Pandas. (2026). *pandas.read_csv*. Pandas Documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html